



Lecture-2

Data Link Layer (LLC and MAC)

List of Questions of Slide - 2

1. What do you mean by frame at LLC sublayer?

Framing refers to the process of partitioning bit stream into discrete units or blocks of data called frame.

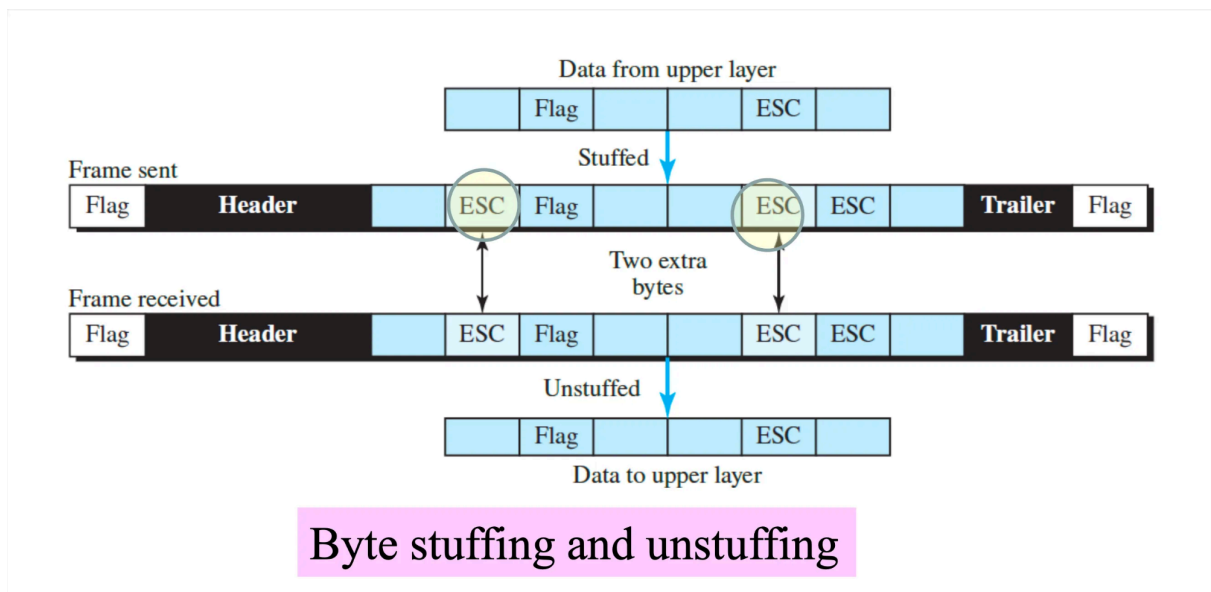
Framing enables sending and receiving machines to synchronize the transmission and reception of data because frames have detectable boundaries.

One common framing procedure involves inserting flag characters before and after the transmitting data message. This helps the receiving device recognize the boundaries of each frame.



Fig.1 Frame format

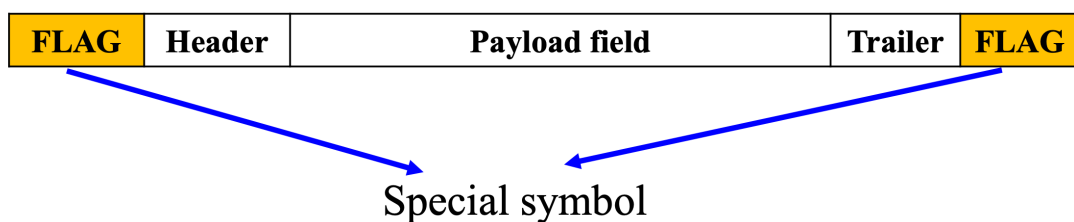
2. Draw byte stuffing and unstuffing technique in character oriented protocol



for better understanding

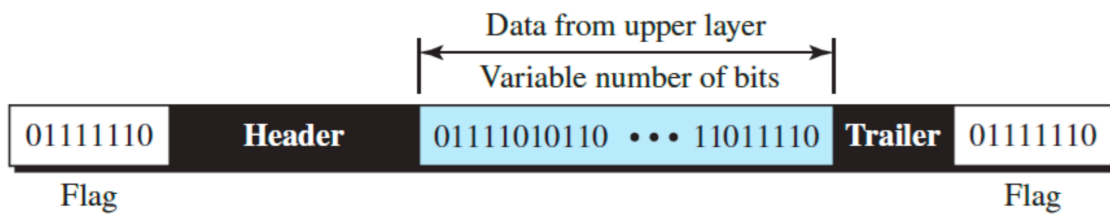
Character-Oriented Protocols (flag byte with byte stuffing)

- In a character-oriented protocol (called **flag byte with byte stuffing**), the component or element of header, data and the trailer composed of character or symbol (character of 8 bits or 1 byte) instead of bit.
- To separate one frame from the next, an 8-bit (1-byte) flag is added at the beginning and the end of a frame.

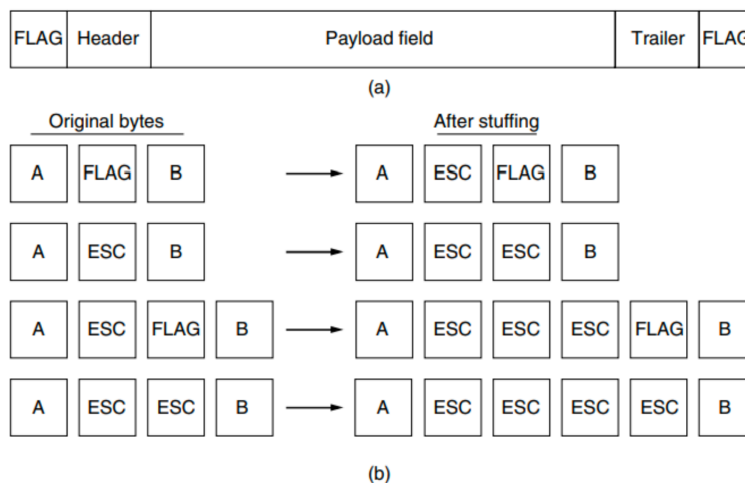


6

- In this way, if the receiver ever loses synchronisation, it can just search for the flag byte to find the end of the current frame. Two consecutive flag bytes indicate the end of one frame and start of the next one.



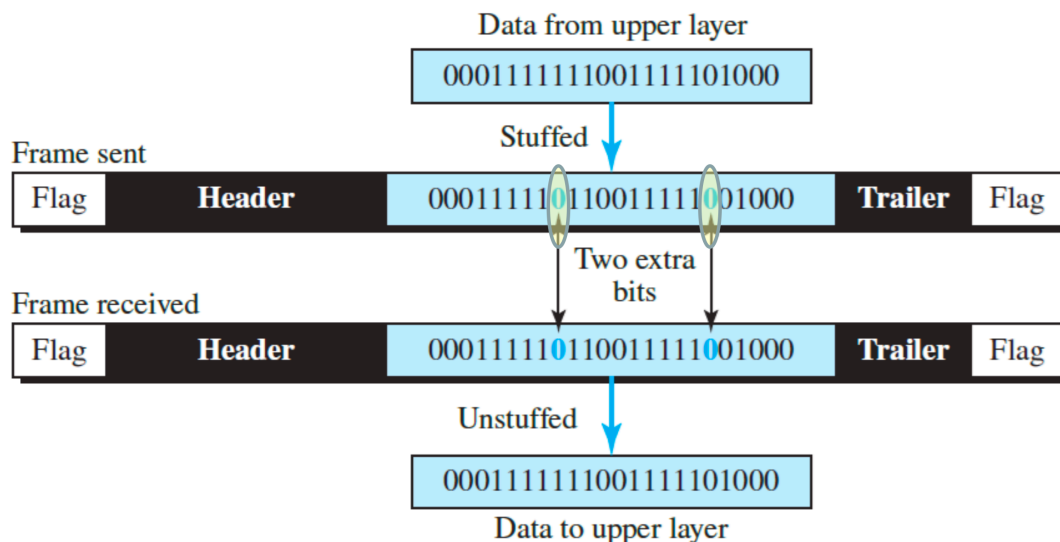
- It may easily happen that the **flag byte's bit pattern occurs in the data**. This situation will usually interfere with the framing. One way to solve this problem is to have the sender's data link layer insert a special escape byte (ESC) just before each 'accidental' flag byte in the data.
- The data link layer on the receiving end removes the escape byte before the data are given to the network layer. This technique is called **stuffing** or character **stuffing**. Thus, a framing flag byte can be distinguished from one in the data by the absence or presence of an escape byte before it.



- The major disadvantage of using this framing method is that it is closely tied to the use of 8-bit characters or byte stuffing. Not all character codes use 8-bit characters for example UNICODE uses 16-bit or 32 bit characters, pixel of each colored image is 24 bits will conflict with 8-bit characters.

3. Draw the idea of bit stuffing and unstuffing under flag bit with bit stuffing

Fig. below gives an example of bit stuffing.



Bit stuffing and unstuffing

12

For better understanding

Flag bits with bit stuffing

- Therefore a third technique is developed to allow arbitrary sized characters called **bit-oriented protocol** (called **Flag bits with bit stuffing**). In this scheme, framing is done at the bit level, so frames can contain an arbitrary number of bits made up of units of any size.
- In a bit-oriented protocol, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on.
- Each frame begins and ends with a special bit pattern, 01111110 (in fact, a flag byte). Whenever the sender's data link layer encounters five

consecutive 1s in the data (payload), it automatically stuffs a 0 bit into the outgoing bit stream. This bit stuffing is analogous to byte stuffing.

- When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically de-stuffs (i.e., deletes) the 0 bit.
- For example, if the user data contain the flag pattern, **01111110**, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110.

4. Compare the best case and worst case of bit and byte stuffing

Comparison of Best Case and Worst Case in Bit and Byte Stuffing :



Both **bit stuffing** and **byte stuffing** affect the frame length depending on the data being transmitted. The more special characters (like flag bytes or bit patterns) in the data, the more the frame size increases.

Byte Stuffing Case

- **Best Case:** If the data contains no special flag bytes, a 100-byte data packet will result in a frame close to **100 bytes**.
- **Worst Case:** If every byte in the data is a flag byte, each one needs to be **escaped** (extra byte added), making the frame nearly **200 bytes long** (double the size).

Bit Stuffing Case

- **Best Case:** If the data contains no long sequences of 1s, the extra bits added will be minimal, keeping the frame close to its original size.
- **Worst Case:** Since bit stuffing adds **1 extra bit** for every **5 consecutive 1s**, the frame size increases by about **12.5%** in the worst case.

In short, byte stuffing can cause a **huge increase** in frame size if the data has many flag bytes, while bit stuffing increases size **more gradually** based on the bit patterns in the data.

5. Compare error correction through retransmission and autonomous error correction



- The term **error control** refers to the process of guaranteeing reliable(নির্ভরযোগ্য) data delivery. There are two main ways to handle errors:

1. The first method, Error Correction through Retransmission:

- **Error correction through retransmission**, involves providing enough information in the data stream so the receiving node can detect an error. Once an error is detected the receiving node can then request the sender to retransmit that unit data.

2. The second method is Autonomous(স্বায়ত্তশাসিত) Error Correction:

- **Autonomous error correction**, involves providing redundant(অপ্রয়োজনীয়) information in the data stream so the destination node can both detect and correct any error autonomously.
- This method is useful in situations where retransmission is costly or slow, such as in satellite communications

6. Example-01

Example-1:

Given, Message bit string $M = 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1$ (10 bits)

Generator bit string $G = 1\ 1\ 0\ 1\ 0\ 1$ (6 bits)

Determine remainder polynomial $R(x)$ and transmitted polynomial $T(x)$.

Ans.

Given, Message bit string, $M = 1\ 0\ 1\ 0\ 0\ 0\ 1\ 1\ 0\ 1$ (10 bits)

Generator bit string, $G = 1\ 1\ 0\ 1\ 0\ 1$ (6 bits)

Determine $R(x)$ and $T(x)$.

$$\therefore M(x) = x^9 + x^7 + x^3 + x^2 + x^0 \quad G(x) = x^5 + x^4 + x^2 + x^0$$

Degree of $G(x)$, $r = 5$.

Message after appending 5 zeros becomes,

1 0 1 0 0 0 1 1 0 1 0 0 0 0 0

The corresponding polynomial, $x^5 M(x) = x^{14} + x^{12} + x^8 + x^7 + x^5$

22

$$\begin{array}{r}
 \text{Divisor } x^5 + x^4 + x^2 + 1 \quad \left| \begin{array}{l} \text{Dividend } x^{14} + x^{12} + x^8 + x^7 + x^5 \\ x^{14} + x^{13} + x^{11} + x^9 \\ \hline x^{13} + x^{12} + x^{11} + x^9 + x^8 + x^7 + x^5 \\ x^{13} + x^{12} + x^{10} + x^8 \\ \hline x^{11} + x^{10} + x^9 + x^7 + x^5 \\ x^{11} + x^{10} + x^8 + x^6 \\ \hline x^9 + x^8 + x^7 + x^6 + x^5 \\ x^9 + x^8 + x^6 + x^4 \\ \hline x^7 + x^5 + x^4 \\ x^7 + x^6 + x^4 + x^2 \\ \hline x^6 + x^5 + x^2 \\ x^6 + x^5 + x^3 + x \end{array} \right. \quad \left[\begin{array}{l} \text{Quotient } x^9 + x^8 + x^6 + x^4 + x^2 + x \\ 12 \quad \begin{array}{r} 79 \\ 72 \\ \hline 7 \end{array} \quad 6 \end{array} \right.
 \end{array}$$

Modulo 2 operation

$$\therefore R(x) = x^3 + x^2 + x \leftrightarrow 0\ 1\ 1\ 1\ 0$$

$$\therefore T(x) = x^5 M(x) - R(x)$$

$$= x^{14} + x^{12} + x^8 + x^7 + x^5 + x^3 + x^2 + x \quad \text{Remainder}$$

23

7. How to avoid single bit error, two bit error, odd number of bits in error, burst error

1. One common type of error is a **single-bit error**, which can be expressed as $E(x) = x^i$ when it affects bit position i . If we select $G(x)$ such that the first and the last term are nonzero, then we already have a two-term polynomial that cannot divide evenly into the one term $E(x)$.

2. For **two bit error**, $E(x) = x^m + x^r$, $m > r$

$$= x^r(x^{m-r} + 1) = x^r(x^k + 1)$$

In this case $G(x)$ has to be chosen such that it does not divide $x^k + 1$. For example if $G(x) = x^{15} + x^{14} + 1$ then $G(x)$ is unable to divide $x^k + 1$ for $k \leq 32,768$.

3. For **odd number of bits in error** then $E(x)$ will contain an odd number of terms. For 3 bit error, $E(x) = x^5 + x^2 + 1$. If any polynomial has an odd number of terms then it does not have any factor like $(x+1)$. If $G(x)$ has a factor of $(x+1)$ it would be unable to divide $E(x)$.

4. For **burst error** of length k then

$$\begin{aligned} E(x) &= x^{k+i-1} + x^{k+i-2} + \dots + x^{i+1} + x^i \\ &= x^i(x^{k-1} + x^{k-2} + \dots + 1) \end{aligned}$$

If the degree of $G(x)$ is r and $r \geq k-1$ then $G(x)$ is simply unable to divide $E(x)$ i.e. error is detected.

✓ For burst error of the size $k = r+1$ is detected with probability,

$$P = 1 - (1/2)^{r-1}$$

✓ For burst error of the size $k > r+1$ is detected with probability,

$$P = 1 - (1/2)^r$$

8. What will happen if $E(x) = G(x)$?

5. If the error sequence and generator are identical then the CRC code will fail to detect error. The probability of such case,

$$P = \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \dots (r+1) \text{ th term}$$
$$= (1/2)^{r+1}, \text{ which is very small}$$

$$E(x) = G(x)$$

$$G = 1001101001$$
$$E = 1001101001$$

9. Give operating principle of sliding window protocol?

Operating Principle of Sliding Window Protocol



The **sliding window protocol** is an improved method for controlling data flow in a network. It enhances the **stop-and-wait protocol** by allowing the sender to transmit multiple frames **without waiting for an acknowledgment**, making data transfer faster and more efficient.

How It Works:

1. Receiver's Buffer Management:

- The receiver **informs the sender** about how much buffer space it has available.

- This prevents the sender from overwhelming the receiver with too many frames.

2. **Continuous Data Transmission:**

- The sender can send multiple frames **without waiting** for an acknowledgment after each one.
- It must ensure that the number of unacknowledged frames does not exceed the receiver's buffer capacity.

3. **Frame Numbering:**

- Each frame is given a **sequence number** so the sender and receiver can track which frames are sent and received.

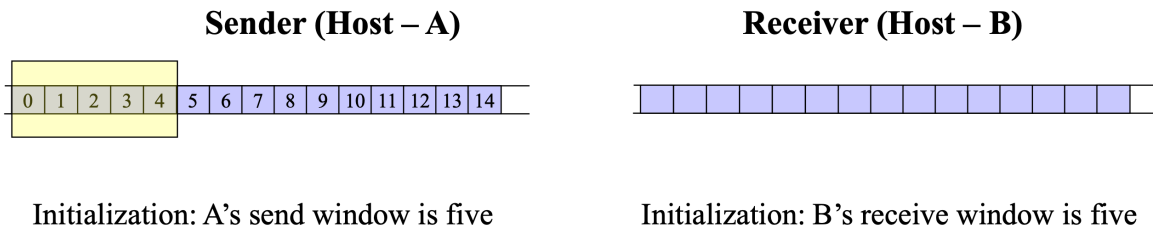
4. **Sliding Window Mechanism:**

- Both sender and receiver use a **window** (a limited set of frames they can handle at a time).
- As acknowledgments are received, the window **slides forward**, allowing the sender to send new frames.

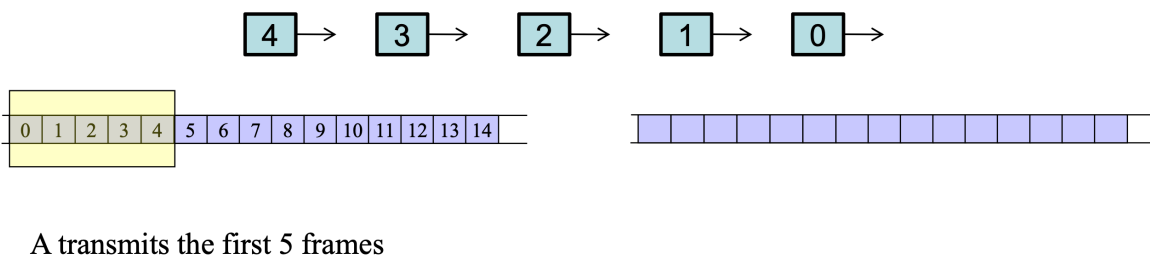
The **sliding window protocol** improves network efficiency by ensuring **continuous data flow** while preventing data loss due to buffer overflow.

10. Draw few step of sliding window protocol for the case of variable size window

Step-1

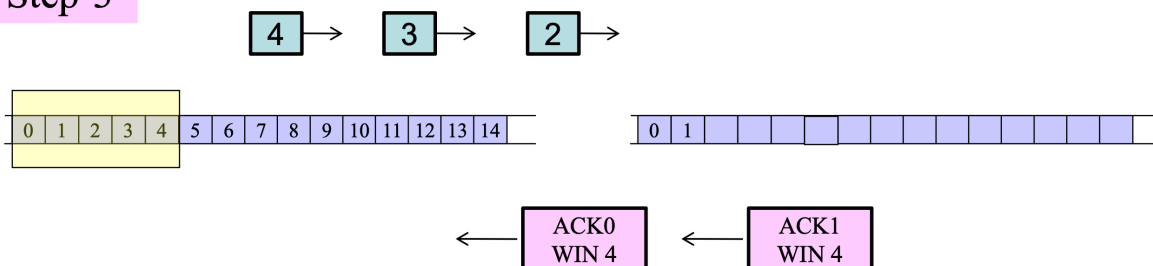


Step-2



50

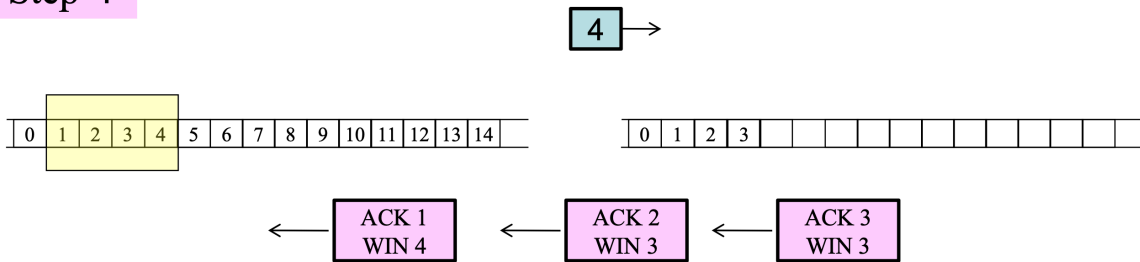
Step-3



A has not received any ACK so it keeps the window size 5 like before

B Receives frame 0 and 1 and sends the corresponding ACK. Each ACK carries a window size of 4. Thus once A receives the ACK for frame 0, it must change its widow size to 4.

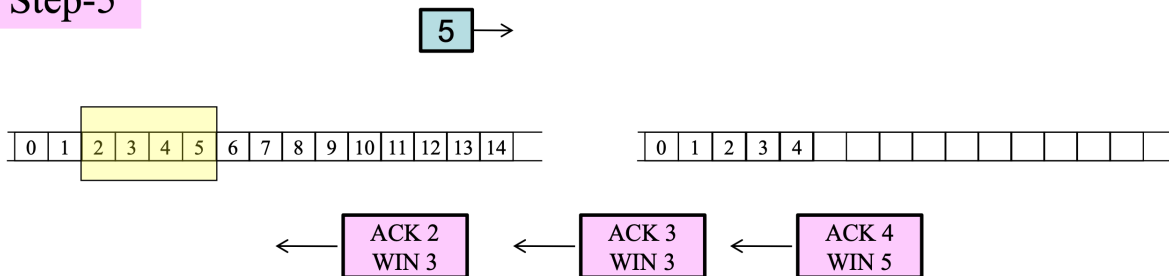
Step-4



A has ACK of frame 0 and advances the left side of its window by one. However, frame 0's ACK had the window size 4 of 4 therefore A cannot advance the right side of its window.

B Receives frame 2 and 3 and sends the corresponding ACK. Each ACK carries a window size of 3. Thus once A receives the ACK, it must change its widow size to 3.

Step-5



A has ACK of frame 1 and advances the left side of its window by one. However, frame 1's ACK had the window size of 4 therefore A can now advance the right side of its window by one and send frame 5

B Receives frame 4 and sends the corresponding ACK with window size of 5. Thus once A receives the ACK, it must change its widow size to 5.

11. Compare pure and slotted ALOHA

Comparison of Pure ALOHA and Slotted ALOHA



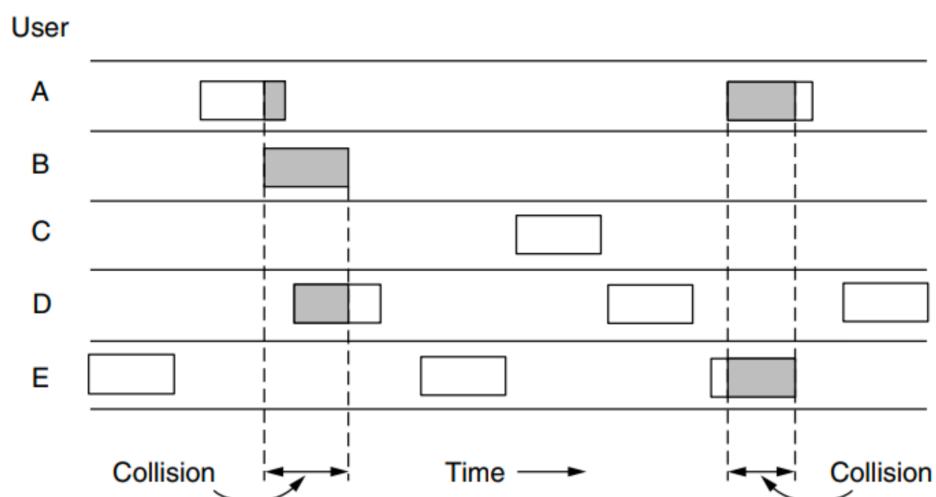
ALOHA is a basic **multiple access protocol** used in communication networks. It has two main types: **Pure ALOHA** and **Slotted ALOHA**.

Feature	Pure ALOHA	Slotted ALOHA
Time Synchronization	No time slots; frames can be sent anytime.	Uses fixed time slots; frames are sent at the beginning of a slot.

Feature	Pure ALOHA	Slotted ALOHA
Frame Transmission	Continuous; a sender transmits whenever data is available.	Discrete; a sender can only transmit at specific time slots.
Collision Probability	High, as frames may overlap at any time.	Lower, as collisions can only occur when two frames start in the same slot.
Efficiency (Max Throughput)	18.4% (at best)	36.8% (at best) – twice as efficient as Pure ALOHA.
Transmission delay	Higher, due to frequent collisions and retransmissions.	Lower, as frames are only sent at synchronized slots.
Usage	Used in older wireless systems and satellite communications.	Used in time-synchronized systems like RFID and early cellular networks.

Key Takeaways:

- **Slotted ALOHA is more efficient** because it reduces collisions by restricting transmissions to time slots.
- **Pure ALOHA is simpler** but has a higher chance of collisions, making it less efficient.



In pure ALOHA, frames are transmitted at completely arbitrary times

63

12. Show that maximum efficiency for pure ALOHA is 0.184 and that of slotted ALOHA is 0.3684

Now the probability of arrival of k-frames during $[0, T_F]$ can be obtained using **Poisson's pdf** like,

$$P_k = \frac{A^k}{k!} e^{-A} = \frac{(\lambda_t T_F)^k}{k!} e^{-\lambda_t T_F}$$

$$P_k = \frac{G^k}{k!} e^{-G}$$

Offered traffic, $A = \lambda_t T_F = G$

Again the probability of no collision is equal to the probability of no arrival within the vulnerable period $[0, 2T_F]$ can be expressed as,

$$P_0 = \frac{(\lambda_t 2T_F)^0}{0!} e^{-\lambda_t 2T_F} = e^{-2G}$$

Therefore the **throughput** of the system,

$$\begin{aligned}\bar{X} &= \text{Offered traffic} \times \text{the probability of no collision} \\ &= (\lambda_t \cdot T_F) \cdot P_0 = G e^{-2G}\end{aligned}$$

Fraction of offered traffic without collision is the carried traffic.

Let us now determine the maximum throughput and the corresponding value of G.

$$\frac{d\bar{X}}{dG} = -2G e^{-2G} + e^{-2G} = 0$$

$$\Rightarrow e^{-2G}(-2G + 1) = 0$$

$$\text{Offered traffic, } A = \lambda_t T_F = G$$

$$\Rightarrow (-2G + 1) = 0$$

$$\Rightarrow G = 1/2$$

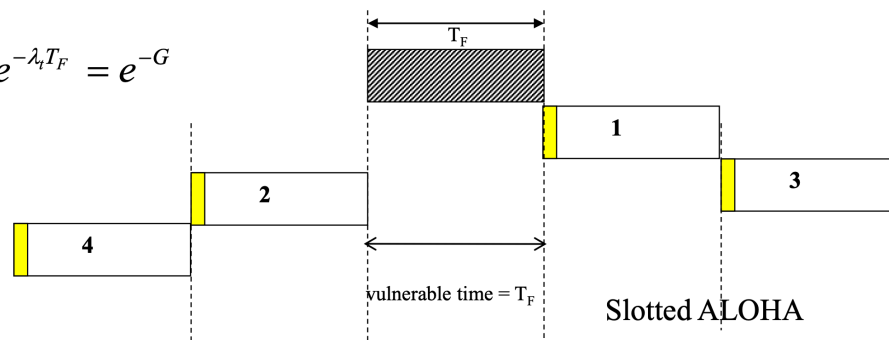
The maximum throughput, $\bar{X} = \frac{1}{2} e^{-2 \cdot \frac{1}{2}} = \frac{1}{2e} = 0.184_{\text{max}}$

The maximum channel utilization of pure ALOHA is only 18.4%

69

For slotted ALOHA the vulnerable period is T_F , therefore the probability of no collision,

$$P_0 = \frac{(\lambda_t T_F)^0}{0!} e^{-\lambda_t T_F} = e^{-G}$$



Therefore the throughput of the system,

$$\bar{X} = G e^{-G}$$

Let us now determine the maximum throughput and the corresponding value of G.

$$\frac{d\bar{X}}{dG} = -G e^{-G} + e^{-G} = 0 \quad \Rightarrow e^{-G}(-G + 1) = 0$$

$$\Rightarrow (-G + 1) = 0$$

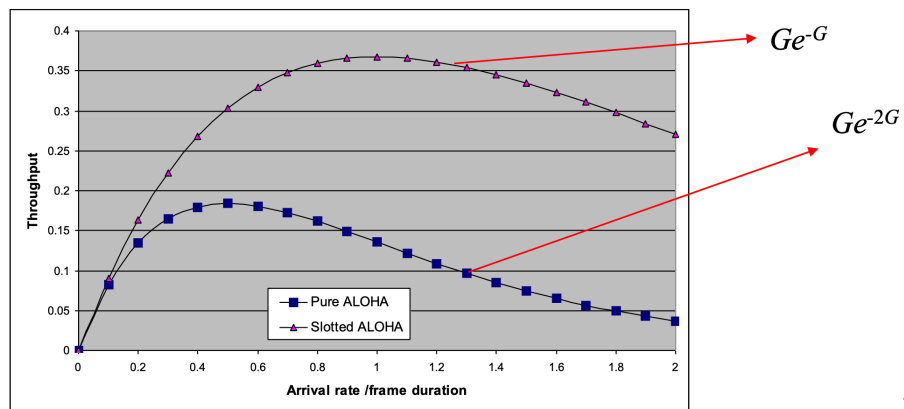
$$\Rightarrow G = 1$$

70

The maximum throughput,

$$\bar{X}_{\max} = e^{-1} = \frac{1}{e} = 0.3684$$

The maximum channel utilization of slotted ALOHA is 36.8%.
The profile of pure and slotted ALOHA is shown in fig. below.



71

13. Example-01

Example-1

A pure ALOHA system uses 60 kbps channel where on an average each terminal generates frame of 1000 bits, where each user on average transmits one frame on every 20 second. How many terminal the system can accommodate? Repeat the job for slotted ALOHA case.

Ans.

The duration of a packet, $T_F = 1000\text{bits}/(60 \times 10^3)\text{bits/sec}$
 $= 16.67 \times 10^{-3} \text{ sec} = 16.67\text{ms}$

The offered traffic by each terminal,

$$G = \lambda \cdot T_F = (1/20) \times 16.67 \times 10^{-3} = 8.335 \times 10^{-4} \text{ Erlangs/users}$$

Let the number of users are N , therefore the total offered traffic,

$G \times N = 1/2e = 0.184 = \text{maximum carried traffic (the maximum \% of utilization of the channel)}$

$$8.335 \times 10^{-4} N = 0.184$$

$$N = 220.75 = 221$$

For slotted ALOHA case,

$$8.335 \times 10^{-4} N = 0.3684$$

$$N = 441.99 = 442$$

14. Compare exhaustive service, gated service, and limited service of token ring protocol

Comparison of Exhaustive Service, Gated Service, and Limited Service in the Token Ring Protocol

In the **Token Ring Protocol**, different service strategies determine how a station transmits data when it captures the token.

Service Type	Description	Behavior	Advantages	Disadvantages
Exhaustive Service	The station transmits all queued packets, including any new packets that arrive	The token is held until the station has sent everything in its queue.	Ensures complete transmission of all pending data.	Can cause long token holding times, leading to delays for other stations.

Service Type	Description	Behavior	Advantages	Disadvantages
	during transmission.			
Gated Service	The station transmits only the packets that were in the queue before capturing the token. New packets arriving during transmission must wait for the next token.	The token is released after transmitting only pre-existing packets.	Provides a fair distribution of token access among stations.	Packets arriving just after token capture must wait for the next cycle.
Limited Service	The station transmits at most k packets , even if more are in the queue.	The token is released after sending k packets, even if the queue has more waiting packets.	Prevents one station from monopolising the token, reducing network congestion.	May lead to delays if k is too low, causing backlog in busy stations.

Key Takeaways:

- **Exhaustive Service** maximizes efficiency for individual stations but can lead to long delays for others.
- **Gated Service** ensures fairness by limiting transmission to pre-queued packets.
- **Limited Service** balances efficiency and fairness by restricting packet transmission per turn.

15. Why wireless network use CSMA/CA instead of CSMA/CD?



Wireless networks use **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)** instead of **Carrier Sense Multiple Access with Collision Detection (CSMA/CD)** because **collision detection is not practical in wireless communication.**

Key Reasons:

1. Collision Detection is Difficult in Wireless Networks:

- In a wireless system, the **transmitting signal is much stronger** than the received signal.
- If a device tries to detect a collision while transmitting, its own signal will overpower the weak incoming signals.
- This makes it **impossible** to accurately detect collisions.

2. Collision Avoidance is More Practical:

- Instead of detecting collisions after they occur, CSMA/CA **prevents collisions before they happen.**
- Devices **listen** for an idle channel before transmitting.
- If the channel is busy, they wait and retry after a random time to **avoid simultaneous transmissions.**

3. Why CSMA/CD Works in Wired Networks:

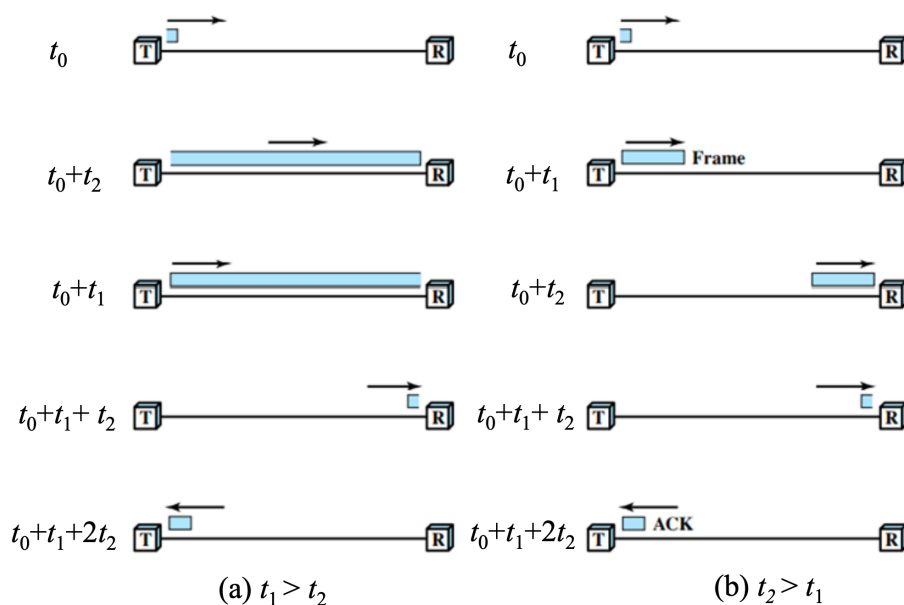
- In wired communication, transmitted and received signals are of similar strength.
- This allows devices to **easily detect** collisions and take corrective actions.
- However, this method does not work well for wireless networks due to the difference in signal strength.

Conclusion:

Wireless networks **cannot detect collisions effectively**, so they use **CSMA/CA** to avoid them, ensuring more reliable communication.

16. Compare utilization of transmission time for smaller and larger frame under stop and wait flow protocol

Let, the **transmission time** (the time it takes for a node to transmit a frame) is t_1 , and the **propagation delay** (the time it takes for a bit to travel from sender to receiver) is expressed as the variable t_2 .



Transmission time is larger than propagation delay

Transmission time is smaller than propagation delay

42

Comparison of Transmission Time Utilization for Different Frame Sizes in Stop-and-Wait Flow Control

The given image illustrates **two scenarios** under the **Stop-and-Wait Protocol** based on the relationship between **transmission time (t_1)** and **propagation delay (t_2)**:

1. Case (a): Transmission Time (t_1) > Propagation Delay (t_2)

- A **larger frame** is used, which means the transmission time is longer than the time it takes for the frame to propagate.
- The sender is actively transmitting for a longer duration, making better use of the transmission link.

- The **utilization is higher** because the sender spends more time sending data instead of waiting for acknowledgments.

2. **Case (b): Propagation Delay (t_2) > Transmission Time (t_1)**

- A **smaller frame** is used, so the transmission time is much shorter than the propagation delay.
- The sender quickly finishes sending a frame but then waits idle for a long time until the acknowledgment arrives.
- The **utilization is lower** because most of the time is spent waiting rather than transmitting.

Key Observations:

- **Larger frames ($t_1 > t_2$) improve link utilization** since the sender transmits for a longer period, reducing idle time.
- **Smaller frames ($t_2 > t_1$) lead to lower utilization** because the sender spends more time waiting for acknowledgments than actually transmitting data.
- **In Stop-and-Wait flow control, the propagation delay significantly affects efficiency**, especially when the transmission time is small.

Conclusion:

To maximize utilization, **larger frames** should be used when possible. However, if the frame size is too large, it may introduce delays in error recovery. An alternative approach, such as **Sliding Window Protocol**, can help improve efficiency by allowing multiple frames to be sent before waiting for an acknowledgment.